

In this article, we will discuss how a poorly written application involving function calls could get hacked, how *gdb* can be invoked in batch mode, and how to write *gdb* scripts and use them to automate the hack. We also look at counter measures for developers to safeguard their applications.



Note: There is no intention to provide cracking techniques; this article is purely for educational purposes. This enables application developers to focus on security-related aspects. It is assumed that readers are acquainted with GCC, C and Linux. We will focus on the Intel platform only, but the idea can be extended to other hardware platforms with a few modifications.

The basic threat makes some functions pass (return success) even if they fail—like even if the password that is provided is wrong. We also touch some nice features of *gdb*, like running *gdb* in batch mode, and *gdb* scripts. Read on, if you find this interesting!

Basic concepts

Usually, a program is divided into a set of functions, each doing a specific task. Once a function is invoked, it has (in general) two characteristics:

- It returns a value—the status of the function call; whether it passed or failed. The caller can take action based on this return value.
- Side-effects—the effects of the function on the global state of the program (like updating a database or a global variable, etc).

Not all functions have side-effects, but almost all functions return a value. The return value normally decides the flow of the program after that call. What if a hacker manages to change the return value of a targeted function? This causes application logic (more precisely, the flow) to be changed to the hacker's advantage—we don't want this!

Consider an application that has a function named *authenticate*. This function takes an input string (*password*), and returns 0 if the password is good—or returns 1 otherwise. Hackers can gain access to the application by changing the return value of *authenticate*.

Victim application: An example

For this article, we have a test application (*hackMe.c*) shown below, where the *authenticate* function is called by *main* to check if the application should continue execution or not. The application exits if the input string passed to it is not equal to 'PASS'. The program command-line argument is passed to *main*, and subsequently to the *authenticate* function.

```
#include<stdio.h>

/*
 * Returns 0 on success, and
 * Returns 1 on failure
 */
int authenticate (char *test)
{
    if(strcmp(test,"PASS") == 0 )
        return 0; /* success*/
    else
        return 1; /* fail */
}

/* USAGE:  . ./hackMe FAIL */
int main(int argc, char *argv[])
{
    int retVal = -1;
    if (argc<2)
    {
        printf ("\n USAGE: %s <PASS|FAIL>", argv[0]);
        exit (-2);
    }
    /* skipping any checks on argv to keep it simple*/

    retVal = authenticate(argv[1]);
    /* WE WILL USE GDB TO ALTER RETURN VALUE OF THIS CALL JUST BEFORE
    COPYING THE RETURN VALUE TO VARIABLE retVal */

    if( retVal == 0)
        printf("\n Authenticated ...program continuing...\n");
    else
    {
        printf("\n Wrong Input, exiting...\n");
        exit (retVal);
    }
    /*
    .
    .
    Rest of the program
    .
    .
    */
}
```

Compile and link the program (without any debug information!) with the following command:

```
gcc hackMe.c -o hackMe
```

Analysing the victim binary

Here are the results of invoking this program with different command-line arguments: